

SWE40006

Software Deployment and Evolution

Lecture 4
Deployment techniques

Lecture overview

This lecture will cover,

- A motivating scenario (Jakarta EE Application)
- Build Numbering and Management
- **Deployment Approaches**
 - **Manual, Scripted, Language** and Model-Based

Manual vs. Automated

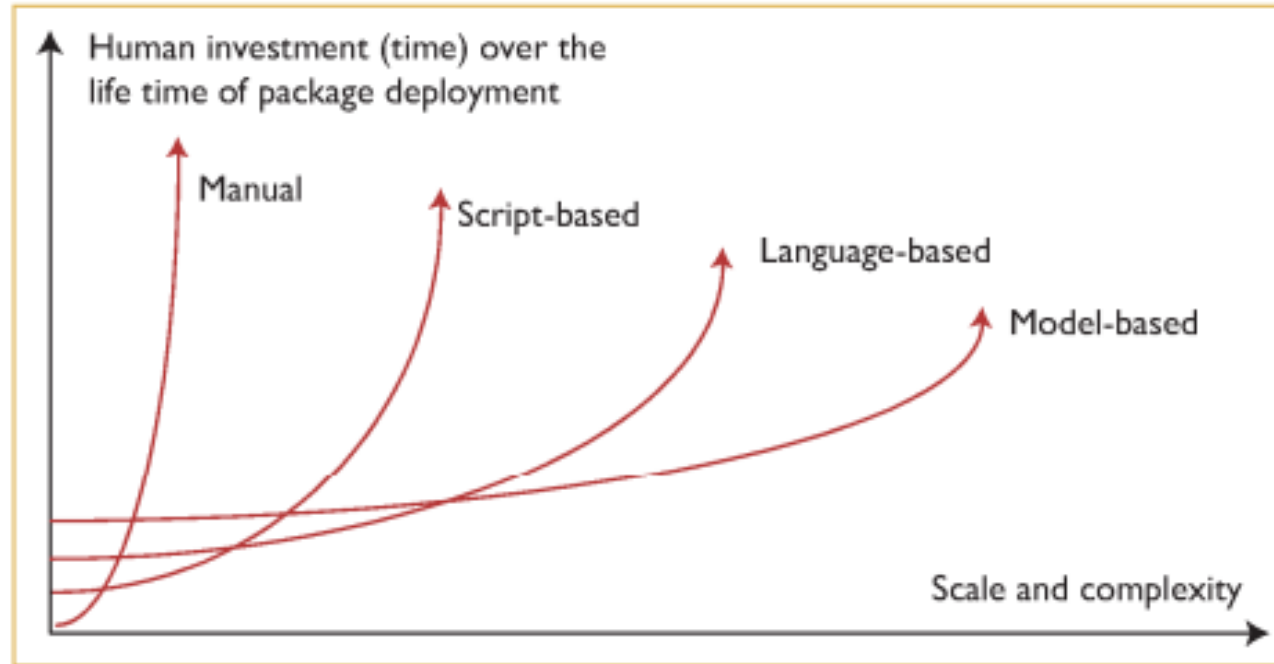


Figure 1. Trade-offs between initial cost (development and learning) and repeated use cost. The level of a tool's automation pushes costs up earlier in the development cycle, but developing tools, learning from them, and creating templates with them pays off as complexity increases.

Source: Talwar 2005

Manual Approach - Example

- Instructions for a trained software engineer (high-level) ...

1. Propagate packages to Web repository
2. Log in to Web server host
3. Unpackage Apache Build and install Apache
4. Edit `httpd.conf`
5. Start Apache
6. Inspect Apache process list
7. Log out of Web server host

- Need more detail in some steps (esp. for new developers)

Scripted Approach

- Partial scripting of some steps is possible...

```

WEB_SERVER=poseidon.cc.gatech.edu #Web repository
WEB_PORT=8080
WEB_DIR="/binaries"
APACHE_ARCHIVE=httpd-2.0.49.tar.gz #Binary Archives
DIR=/usr/local #Installation Directory
APACHE_INSTALL_HOME=$DIR/httpd-2.0.49
cd $DIR
if [[ ! -d $APACHE_HOME ]] : then
  wget
  $WEB_SERVER:$WEB_PORT/$WEB_DIR/$APACHE_ARCHIVE
  tar -xzf $APACHE_ARCHIVE
  cd $APACHE_INSTALL_HOME
  ./configure > /dev/null
  make > /dev/null
  make install > /dev/null
  cd $DIRECTOR
  rm -r -f $APACHE_INSTALL_HOME
  rm $APACHE_ARCHIVE
fi

```

You will still have to check that the system works end-to-end manually.

Is it worth the effort?

How about data migration?

Language-Based Approach

- Scripting is great, but operates at a very low level
- If we can abstract some aspects -- we can use high-level languages
- This allows for better reuse across environments
- It will be shorter than scripts -- if the correct abstractions (i.e. functions/classes/modules) are available
- Compilers can be used to help check many aspects of the deployment program
- Improved quality of testing of the deployment process is possible

Language Based Deployment

- Some languages exist for this purpose
 - Example: Smart Frog
 - <http://www.smartfrog.org/>
 - <https://en.wikipedia.org/wiki/SmartFrog>
 - Ant, Maven and Capistrano have been extended for deployment
 - Ant and Maven are also declarative languages that broadly fall into this category

Language Based Deployment - Smart Frog

- A sample Smart Frog Deployment program
- This is a java-like language
- Once a whole lot of abstractions/templates are created, then a deployment can be composed on these

```

ApacheInstaller extends GenericTarInstaller {
  webServerHost "poseidon.cc.gatech.edu";
  tarLocation "/binaries";
  file "httpd-2.0.49.tar.gz";
  installScript extends vector
    - ATTRIB downloadApache;
    - ATTRIB cdApacheHome;
    - ATTRIB unTarApache
    - ATTRIB configureScript;
    - ATTRIB makeScript;
  cdApacheHome extends concat
    - "cd ";
    - ATTRIB installLocation;
  untarApache extends concat
    - "tar -xzf ";
    - ATTRIB file;
  configureScript "./configure";
  makeScript "make install";
  actions:downloadApache extends Downloader {
    url extends concat {
      - "http://";
      - ATTRIB webServerHost;
      - ":";
      - ATTRIB webServerPort;
      - ATTRIB tarLocation;}
    toLocalFile - ATTRIB installLocation; }}
sfConfig extends ApacheInstaller;

```

Source: Talwar 2005

Language Based Deployment - Maven

Maven Deployment

```

<settings>
  ...
  <servers>
    <server>
      <id>ssh-repository</id>
      <!--
      | Change default file/dir permissions
      -->
      <filePermissions>664</filePermissions>
      <directoryPermissions>775</directoryPermissions>
      <configuration>
        <sshExecutable>plink</sshExecutable>
        <scpExecutable>pscp</scpExecutable>
      </configuration>
    </server>
  </servers>
  ...
</settings>

```

Maven is a declarative language (like Ant)

Language Based Deployment - Ant

```
<target name="deploy-local" depends="webapp">
  <property name="tomcat.local" value="c:\tomcat4"/>
  <copy todir="${tomcat.local}/webapps/${project}">
    <fileset dir="${webapp}" />
  </copy>
...
...
<target name="deploy-live" depends="webapp">
  <property name="live.server" value="rvasa@genius.com"/>
  <property name="live.tomcat" value="/usr/java/tomcat"/>
  <exec executable="rsync">
    <arg line="rsync -e ssh -Cavz ${webapp.rsync}/* $
{live.server}:${live.tomcat}/webapps/${project}" />
  </exec>
</target>
```

Ant Deployment

Ant is a declarative language
(but extended via Java, where new
targets can be defined)

Language Based Deployment - Capistrano

- Capistrano is a Ruby-based language that can be used for deployments
- Supports extensions, just like Ant and Maven
- Extensions are developed in Ruby

```

set :application, "my-webapp"
set :scm, :none
set :deploy_via, :copy
set :repository do
  fetch(:deploy_from)
end
set :war, "/home/andy/webapps/my-
webapp/target/my-webapp.war"
role :webserver, "testserver"
set :tomcat_home, "/usr/share/
tomcat55/"
set :tomcat_ctrl, "/etc/init.d/
tomcat55"
set :user, "testuser" # the user
to run remote commands as
set :use_sudo, false
set :deploy_from do
  dir = "/tmp/"
  prep_#{release_name}"
  system("mkdir -p #{dir}")
  dir
end
set :deploy_to do
  "/u/apps/#{application}"
end
...

```

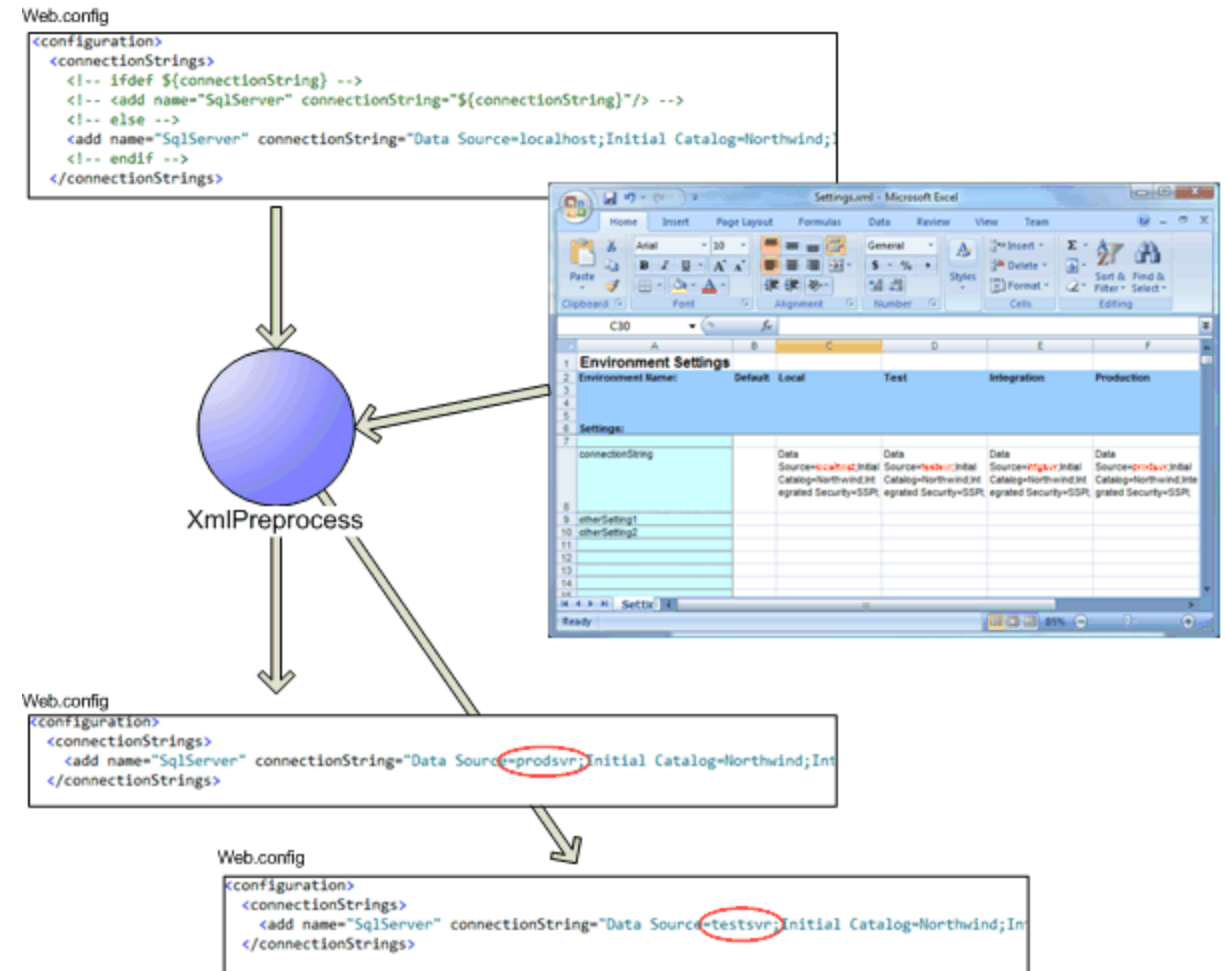
Language Based Deployment - MS Build

- MS Build is an Ant like language to help declaratively build and deploy .NET applications
- Mostly generated by Visual Studio -- rarely customised by hand (in practice)

```
<ItemGroup>
  </AssemblyAttributes>
  <AssemblyAttributes Include="AssemblyTitle">
    <value>MyCompany MyWeb</value>
  </AssemblyAttributes>
  <AssemblyAttributes Include="AssemblyDescription">
    <value>Corporate Site</value>
  </AssemblyAttributes>
  <AssemblyAttributes Include="AssemblyCompany">
    <value>MyCompany</value>
  </AssemblyAttributes>
  <AssemblyAttributes Include="AssemblyCopyright">
    <value>Copyright MyCompany 2005</value>
  </AssemblyAttributes>
</ItemGroup>
```

Language Based Deployment - MS Build

- **xmlPreProcess** is a tool to help generate scripts for multiple environments from MS Build scripts
- <https://github.com/lorenh/xmlpreprocess>



Language Based Deployment

- Windows software can be scripted for installation using WiX
- WiX is a declarative language -- based on XML (like Ant)

```
<Component Id='SqlComponent' Guid='YOURGUID-D8C7-4102-BA84-9702188FA316'>
  <sql:SqlDatabase Id='SqlDatabase' Database='Foobar'
User='SQLUser' Server='[SQLSERVER]'
  CreateOnInstall='yes' DropOnUninstall='yes'
ContinueOnError='yes'>
    <sql:SqlScript Id='CreateTable' BinaryKey='CreateTable'
ExecuteOnInstall='yes' />
  </sql:SqlDatabase>
</Component>
```

WiX script snippet for DB setup

Language Based Deployment

- Ant and Maven are used extensively in Java projects
- Capistrano is used in projects that use Ruby and for Ruby on Rails
- Python has Fabric
- Ansible and Puppet are DSLs (domain-specific languages) for scripting
- .NET has WiX (a declarative language for Windows based software installations)
- Linux has apt-get or choco (Windows port).
- Deployment of .NET projects often require the use of multiple languages in complex environments

Deployment - Puppet

- Manages configuration declaratively
- Users describe the owner resources (and state) in a Puppet manifest file
- Puppet then moves resources and achieve required state (e.g. running, installed)

```
package { 'openssh-server':
  ensure => installed,
}
file { '/etc/ssh/sshd_config':
  source => 'puppet:///modules/sshd/ssh_config',
  owner  => 'root',
  group  => 'root',
  mode   => '640',
  notify => Service['sshd'], # sshd
                                will restart whenever you
                                edit this file.
  require => Package['openssh-server'],
}
service { 'sshd':
  ensure => running,
  enable => true,
  hasstatus => true,
  hasrestart => true,
}
```


Deployment - Ansible

- Python-based easy-to-learn tool for configuring and managing systems
- Can tackle multi-node deployment & ad-hoc task execution
- Manages nodes over ssh
- Uses YAML (i.e. declarative specification)
- The descriptions are called Playbooks (which are expressed in YAML)

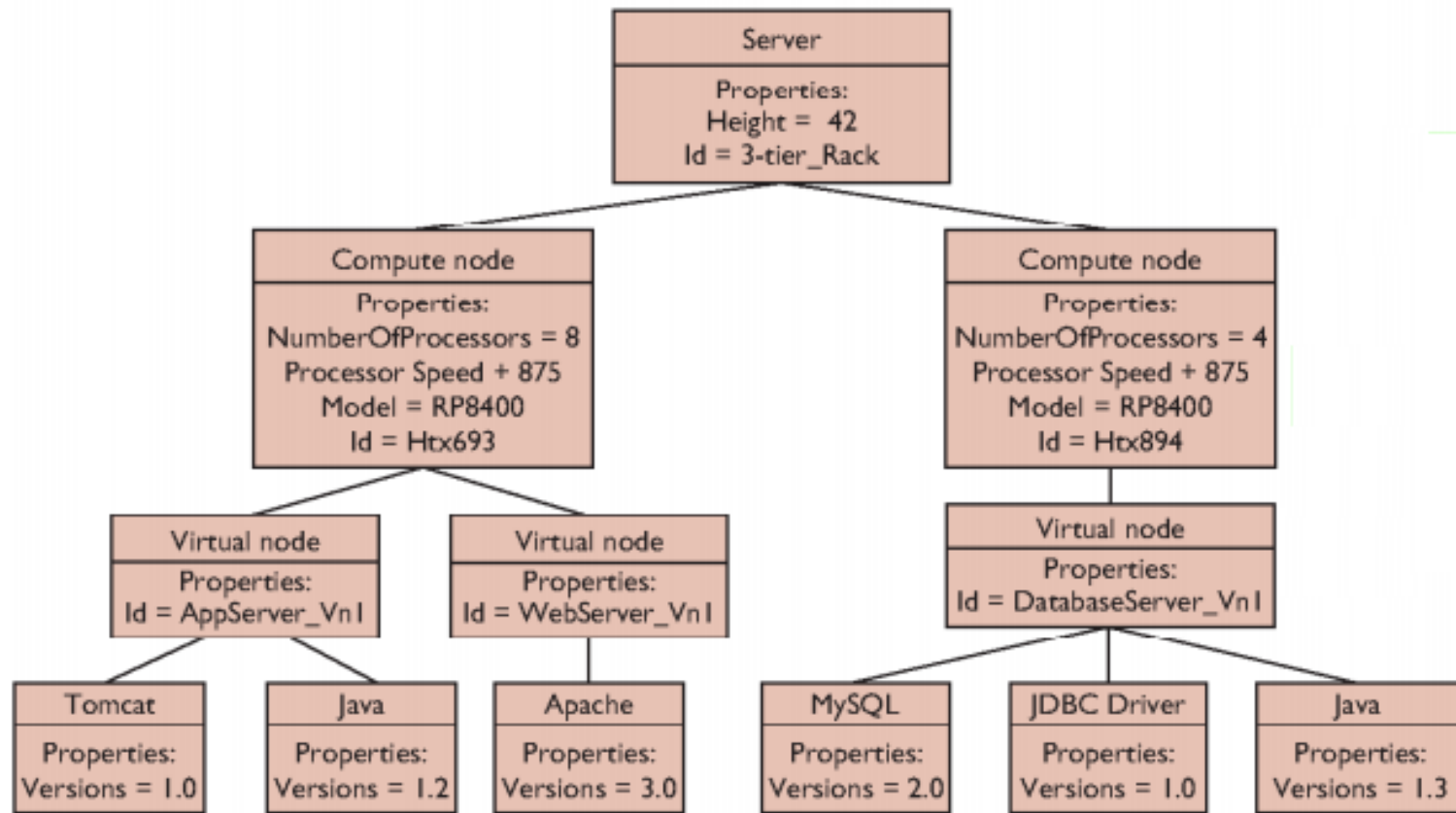
Lecture overview

This lecture will cover,

- A motivating scenario (Jakarta EE Application)
- Build Numbering and Management
- **Deployment Approaches**
 - Manual, Scripted, Language and **Model-Based**

Model-Based Deployment

- This assumes that you can create a model of your environment first -- then map deployment information to that model
- Models still have to be mapped into a language or a script for execution



Model-Based Deployment

- Key advantage here is that each “Node” can have a life-cycle attached to it -- can also attach workflows
- Very handy for monitoring and communicating deployment structure
- Models still require scripts and potentially a deployment language in order to execute the deployment activity i.e. it is a lot more work

Model-Based Deployment

Research focus shows that models need a lot of precise information added to them ...

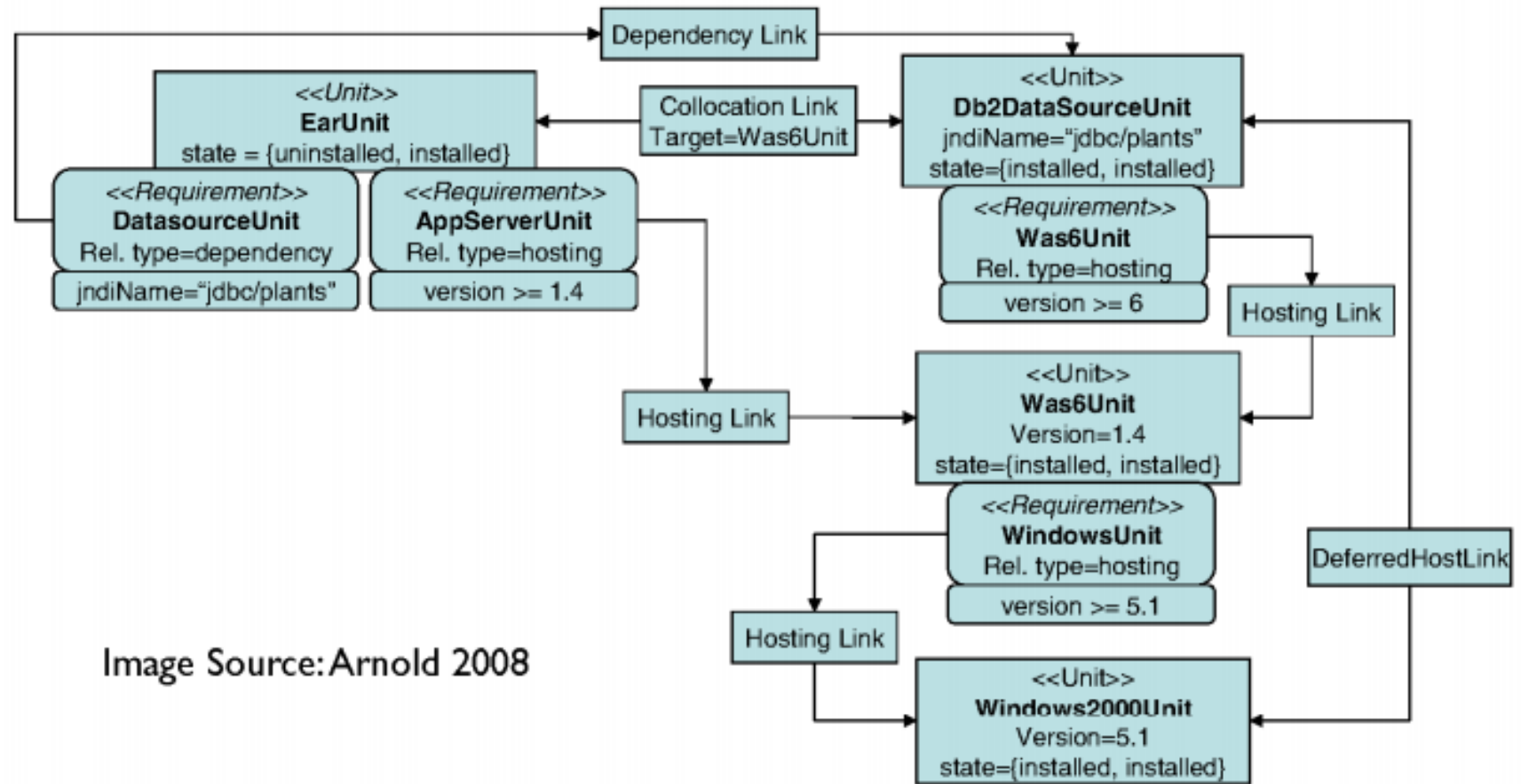
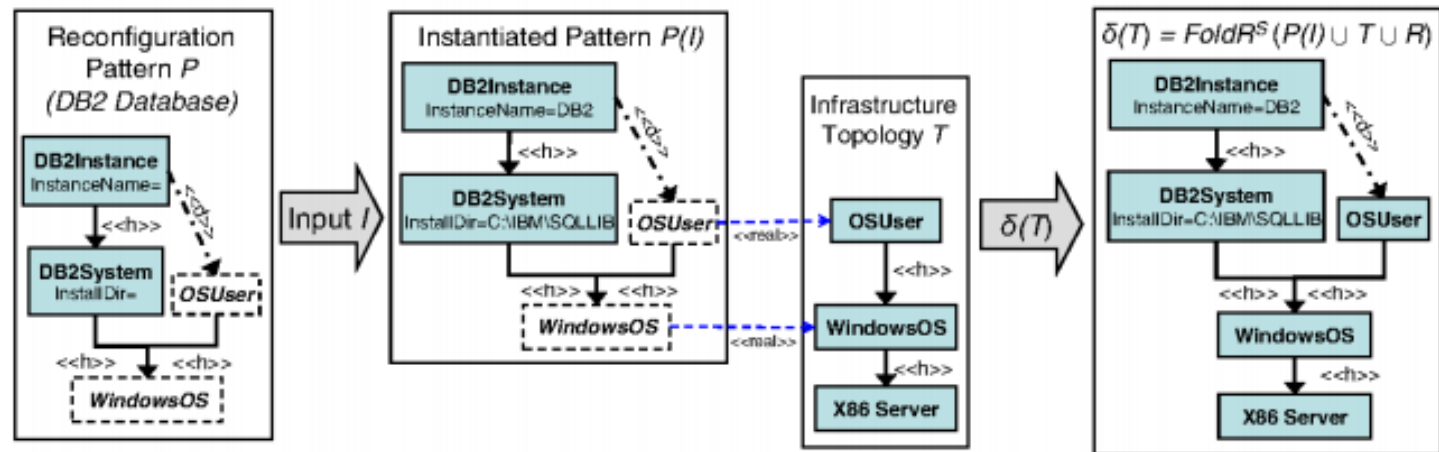
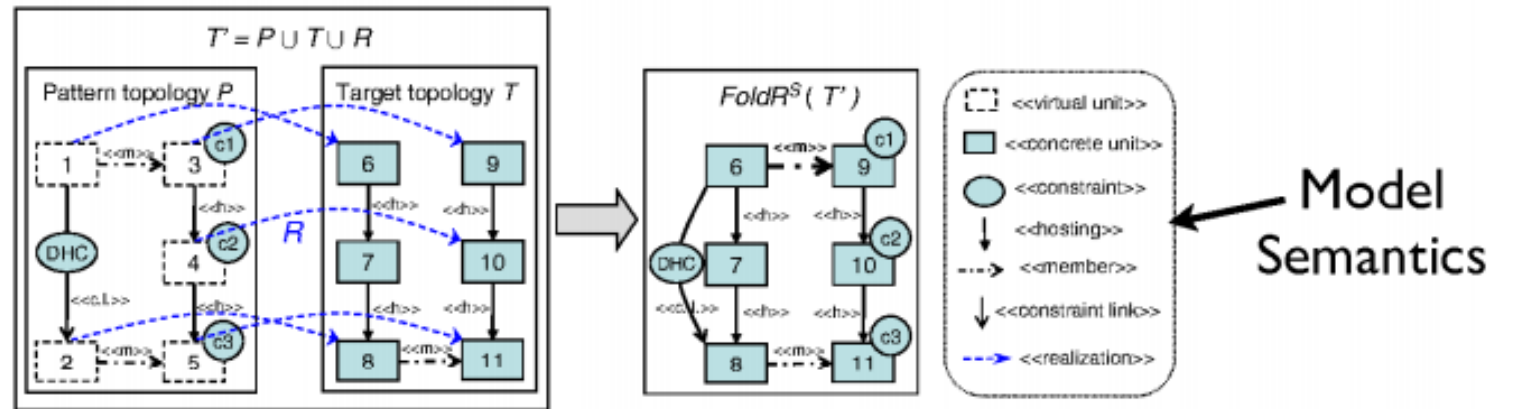


Image Source:Arnold 2008

Model-Based Deployment (research)

- Once models are properly specified, some level of automatic reconfiguration is possible...



Complexity of Deployment

Some examples..

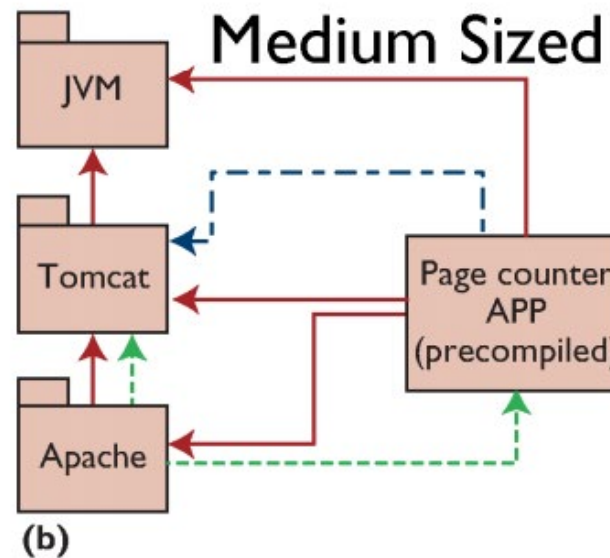
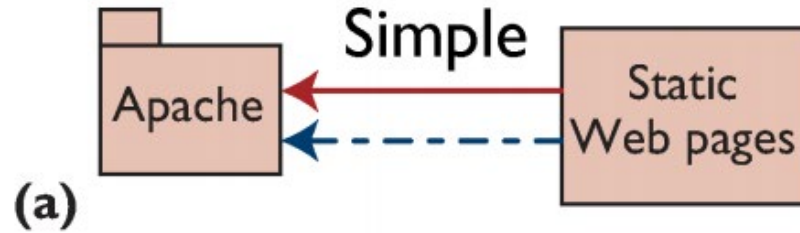


Image Source: Talwar 2005

Complexity of Deployment

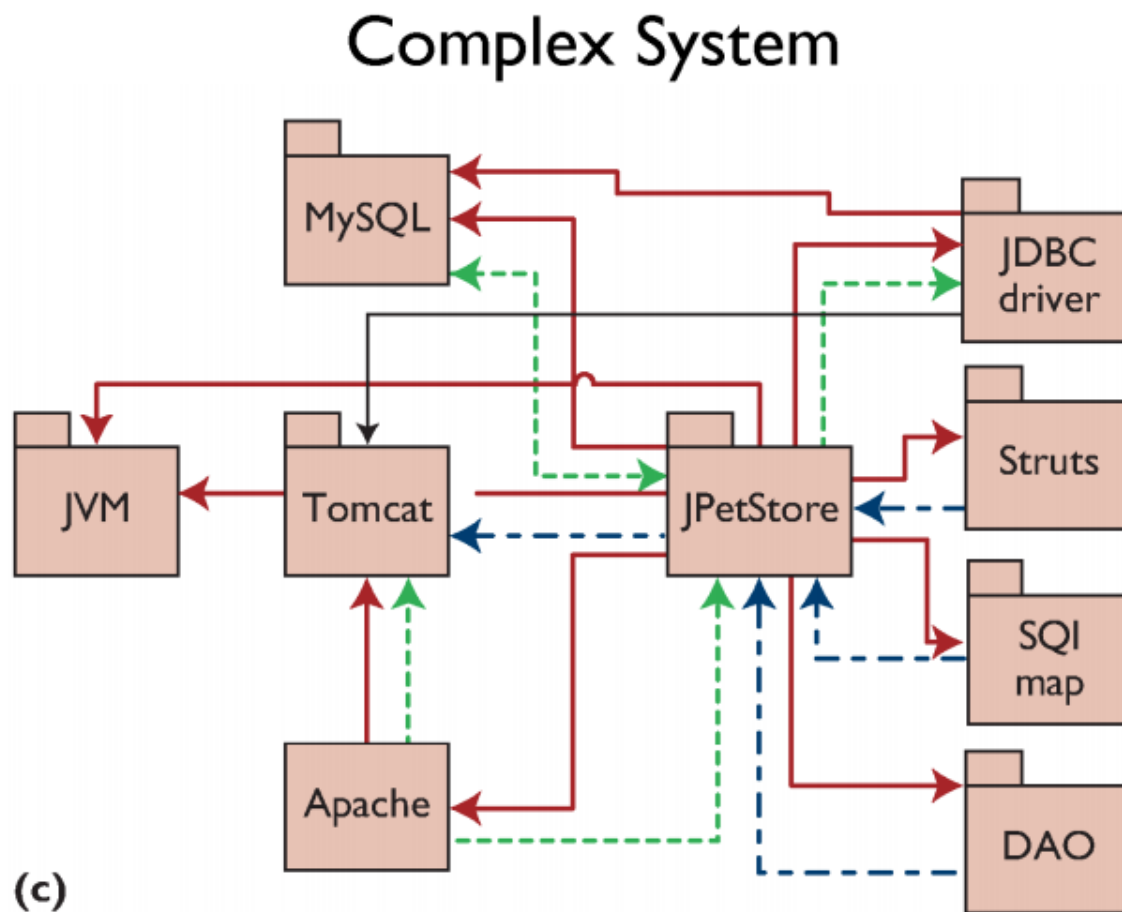
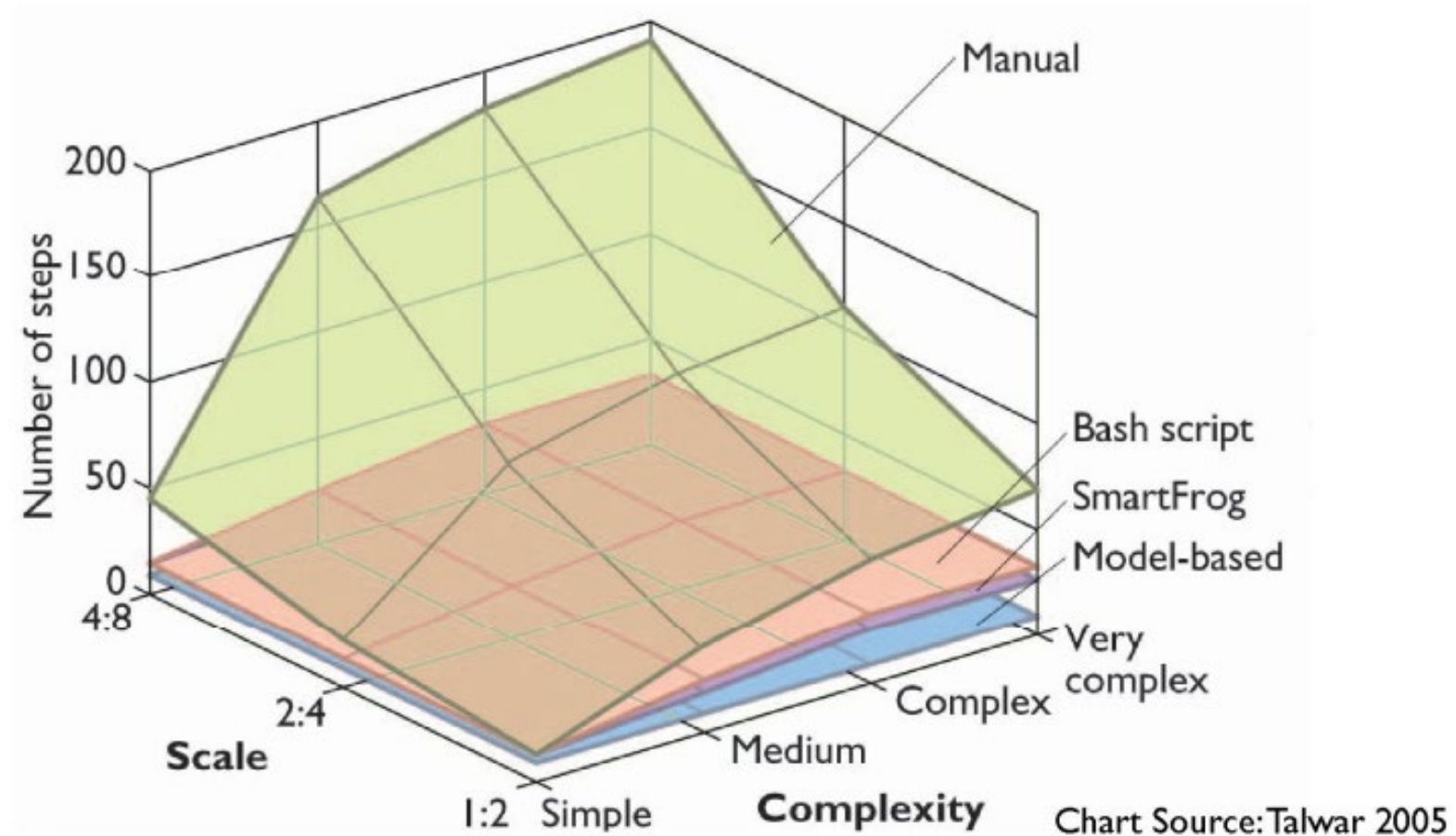


Image Source: Talwar 2005

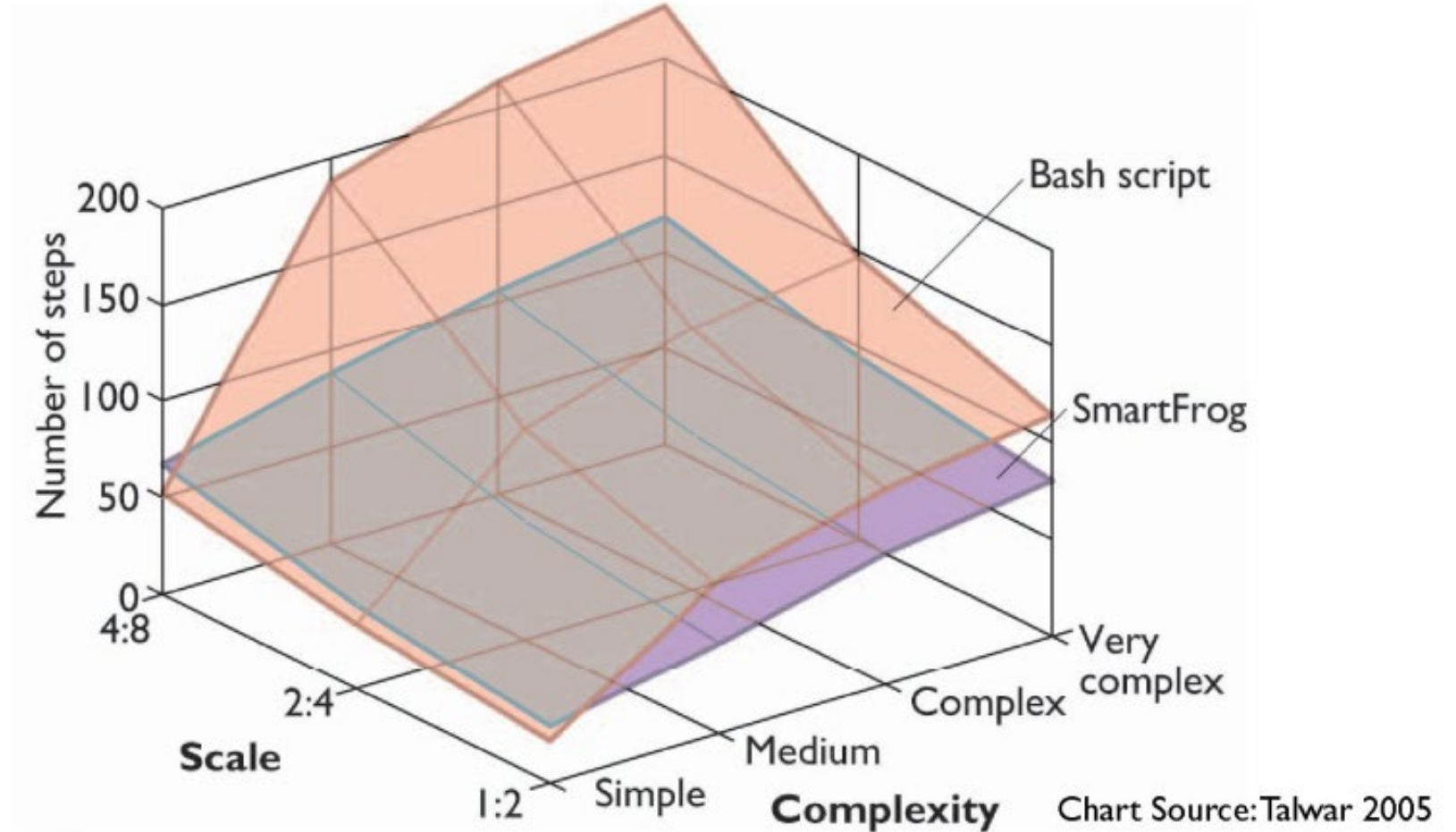
Manual vs. Automation

- Manual is acceptable for simple configurations, but the number of steps will increase for complex configurations



How Much Automation?

- Automation depth is also dependent on the number of steps...



Impact of Load Management

- What if there is a slight increase in load for a short period of time and then it dies down?
- Examples:
 - VCE Student Result server (once a year peak)
 - Fire department server (Bushfire season)
 - AFL Footy Game Servers (weekends)
 - Graduate Recruitment (Twice a year)
- What is needed?
 - An increase in physical servers to meet demand for short period of time -- followed by removal of servers and all installed software

Frequency of Deployments - Issues

- How often is too often?
- How much change can we realistically control?
- Tracking defects and changes
 - The choice of manual, script, language or deployment is influenced by the frequency of deployment

Reading

- Talwar 2005: Approaches for Service Deployment, IEEE Internet Computing, April 2005.
 - [https://www.researchgate.net/publication/3419716 Approaches for Service Deployment](https://www.researchgate.net/publication/3419716_Approaches_for_Service_Deployment)

SWE40006

Software Deployment and Evolution

Lecture 4b
Software Deployment Planning and Support

Software Deployment - Story so far...

- We covered the activities within software deployment
- We also covered the methods that can be applied to deploy (manual vs automated) -- and selection drivers
- In this lecture, we will look at a generic planning frameworks

Related Learning objectives

1. Describe the activities in software deployment and apply these in a problem context.
2. Apply knowledge of software environments to plan development and deployment.
3. Describe and classify issues that drive software maintenance.

Lecture overview

This lecture will cover,

- A deployment planning framework
- Data migration

Situation...

- You will need to deploy a PHP Web Application to a production environment based in US
- We have already covered:
 - The activities in a deployment and the broad execution method
- We need a bit more information to plan the deployment and ensure that it can be properly supported
- Deployments need to be (ideally) tightly planned and executed -- with multiple fallback scenarios

Why is a plan needed

- Most deployments involve an update/upgrade to existing systems
 - Failure or Downtime will impact on
 - Productivity (imagine 500 idle employees for 2 hours @ approx. avg. salary of \$35/hr)
 - Reputation (if Windows 10 upgrade fails)
 - Good enough working software wins over unavailable software
- Ad-Hoc unplanned deployments can be risky
 - Consider a data corruption scenario (career limiting move?)

GMail - Deployment Problem (Sep 09)

- GMail attempted a routine update -- it took out e-mail for millions of people for nearly 2 hours
- See Google's own blog explanation at:
- <https://gmail.googleblog.com/2009/09/more-on-todays-gmail-issue.html>
- Google infrastructure is a complex system
- Resilient most of the time -- but when it fails it can cause cascading ripples taking down many systems
- What is a complex system?
- See: http://en.Wikipedia.org/wiki/Complex_systems

Framework for Planning

- The following information is needed:
 - Deployment Model (for scope of work)
 - Constraints (all the below are inter-related)
 - Temporal
 - Environment and Infrastructure
 - Automation
 - Communication
 - Change pattern/properties
 - Impact of deployment

Software Deployment Model

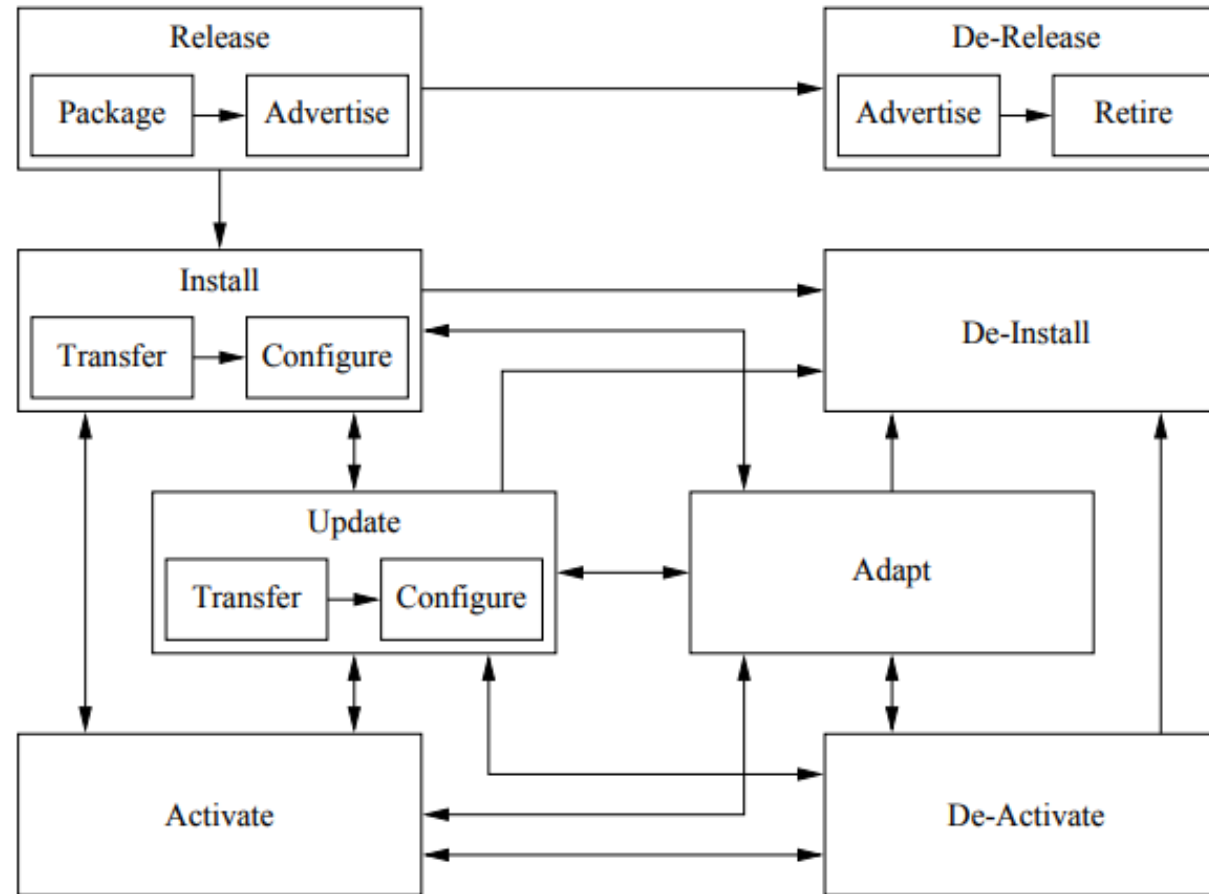


Figure 1: Activities of the Software Deployment Process.

This model will provide the scope of work i.e. "What"

<https://www.ics.uci.edu/~andre/papers/T3.pdf>

Temporal Constraints

- Temporal = “Time related”
- The following temporal information is needed:
 - How long will a deployment take (for a certain environment)?
 - How often do we need to deploy (into a certain environment/infrastructure)?
- Downtime and Availability
 - What is acceptable?
 - What is required? What was promised?

Environment/Infrastructure Constraints

- Operating Systems
- Programming languages and culture
- **Bandwidth available vs needed**
- Security, Safety, Reliability and Legal aspects
- Where is the infrastructure?
 - Fully managed / Outsourced infrastructure?
 - Who owns it?
 - Who manages it? Who is responsible for license management?

Automation Constraints

- Level of automation
 - Manual - Scripted - Language – Model-driven
- Tooling and Experience of team performing the deployment
- Availability of security permissions, tools and people to perform deployment
 - An issue when environment is controlled by an external hosting provider

Deployment Communication

- Planning will need to take into consideration:
 - Who needs to be notified?
 - Who can send out notifications? (Authorisation for communication)
 - What will be communicated?
 - Support during/after the deployment process
 - How much notice is needed for downtime?
- Good rule of thumb to follow:
 - *"Say nothing externally till you have a successful staging deployment"*

Monitoring - Considerations

- The software has to be designed to be monitored
- Logs should provide sufficient information to assess the state of software
- Exceptions must be logged (always)
- Errors must be logged (always)
- *Logs generated must support easy archival (important)*
 - Generally done by ensuring log files are a fixed size (e.g. 1MB)
- Programmers need detailed information to replicate and fix defects -- worth the cost of logging

Framework for Planning - Where are we?

- The following information is needed:
 - Deployment Model (for scope of work)
 - Constraints (all the below are inter-related)
 - Temporal
 - Environment and Infrastructure
 - Automation
 - Communication
 - **Change pattern/properties**
 - impact of deployment

Change Patterns and Properties

- Anticipated changes that require adaptation/update (as opposed to unanticipated changes)
 - Examples 1: Load increases around Christmas and there is a need for additional servers
 - Example 2: Legislative changes (like tax rates)
- How does the system adjust to change
 - Proactive (designed to adapt -- can do so automatically)
 - Reactive (triggered after external change -- requires developer input)

Dimensions of Change

- The following are the dimensions that need to be considered:
 - What is the set of entities that will change?
 - What is the granularity of change?
 - Entire system, some components, configuration files etc.
 - What is the impact of the change on the system
 - How often do these changes take place? (frequency)
 - What is the typical size of change?
 - Duration of change event

Framework for Planning - Where are we?

- The following information is needed:
 - Deployment Model (for scope of work)
 - Constraints (all the below are inter-related)
 - Temporal
 - Environment and Infrastructure
 - Automation
 - Communication
 - Change pattern/properties
 - **impact of deployment**

Deployment Impact

- What are all the systems that will be impacted by this deployment?
- Plan must also incorporate:
 - User/Support documentation
 - User/Support staff training
 - Support infrastructure
 - Monitoring of system post-deployment
- What if deployment fails or causes data corruptions
- What if deployment causes other (working) systems to fail?

Deployment Impact - Monitoring

- A deployment plan needs to take into consideration how the software will be monitored once deployed
- This monitoring will ensure you can actually support the system to promised service level agreement (SLA)

Monitoring Software - Approaches

- Broadly there are two types of monitoring:
 - *Synthetic* monitoring
 - We run scripts to check the performance (or) integrity of deployment
 - Needed soon after activation to ensure that system is functioning to expectations
 - *Passive* monitoring (or real-usage monitoring)
 - Needed for supporting product
 - We monitor traffic and logs while system is in use

Deployment Planning

- Deployment must be planned and verified to work
- Ensure that people performing the deployment have the skills
- Updates may require to demonstrate a tested roll-back plan
- Impact of the deployment is well understood
 - Especially, what if it fails?
 - What if the worst case scenario does happen?
- Poorly planned and executed deployments are very risky -- potentially career limiting

<https://docs.oracle.com/cd/E19263-01/817-5764/architecture.html>

Lecture Overview - Where are we?

- In this lecture we will cover,
 - A deployment planning framework
 - **Data migration**

Data Migration

- Most deployments involve an update/adaptation of existing software systems
- Updates may require 'data migration'
- Constraints that you will need to work with:
 - Data exists on a production environment
 - Data may be dispersed and managed by a range of different data management systems (RDBMs, XML databases, Text files, Custom binary files etc.)
- How should you migrate the data in this situation?

Data Migration - Change

- Data always has an underlying schema
- Data migration will be triggered by change
- Changes to an abstraction (at any level) can be:
 - Addition (e.g. Create new table)
 - Removal (e.g. Drop table)
 - Modification (e.g. Alter table -- modify the data type of a column from varchar to a CLOB)
- What is a safe strategy for data migration, when the schema changes?

Data Migration - Approaches

- Change will alter the underlying 'schema' of a database
- Approaches available when migrating data:
 - *Modification of schema*: The existing database is modified via commands provided to the DBMS (may cause information loss)
 - *Versioning of the schema and data*: We keep the old version -- create new and migrate the data across (safe -- but expensive)
 - Limitation is size of database and available resources (e.g. terabyte databases)

Data vs Information

- Data at the basic level is bytes with a type (i.e. int/ string/date)
- Information is 'what it means to the software application' = Data + Meaning from domain
- We need to consider the 'domain level' meaning of a certain abstraction and consider changes at that level
- Examples:
 - What is the change to the Student entity between Versions 2.4 and 2.5

Data Migration - Strategy

- We need to migrate information -- we must preserve the domain meaning (semantics) when data is migrated
- Why?
 - This ensures that business rules and any transformations are persisted properly
- How can we do this?
 - Version x provides an API for data retrieval at the 'conceptual/domain' abstraction level
 - Version $x+1$ will call the API from previous version to retrieve data and then saves it

Data Migration

- A software application should provide an API for programmers to pull meaningful data from inside the various data sources
 - This ensures that the new version can import data properly from the old
- What is the alternative!
 - Direct modification of database followed by an import (e.g. via SQL alter statements + import scripts)
 - Can work for really small and simple changes - unrealistic and unmanageable on large projects

Issues with direct migration

- ADD scenario: Add column to an existing table in Version 2. The table has 100K rows already.
 - All current data will now have a potential NULL value (or) some default has to be made up to ensure that all queries work properly.
 - What if this column contains data that drives application logic or workflows
- MODIFY scenario: Data type of existing column has been modified. E.g. Integer to a Float
- REMOVE scenario: Existing column removed. Has a significant impact on testing.

Issues in Data Migration

- REMOVE scenario: Existing column removed
 - Has a significant impact on testing (make sure nothing breaks)
 - Are there external applications that rely on this bit of information?
- MOVE scenario: A column or rows of information has been moved from one table to another
 - Often justified to improve performance (e.g. remove a join), but impacts testing
- SHUFFLE scenario: Data is moved across multiple locations.